

Servidor Node.js com Login e Cadastro

Este material explica passo a passo como configurar um servidor Node.js utilizando Express, SQLite3 e Bootstrap, com páginas separadas para Login e Cadastro. No final, há um desafio para os alunos expandirem o sistema.

■ Passo a passo do arquivo server.js

```
// 1) Importar as bibliotecas necessárias
const express = require("express");
const sqlite3 = require("sqlite3").verbose();
const bcrypt = require("bcryptjs");

// 2) Configurações básicas do servidor
const app = express();
const PORT = 3000;
const db = new sqlite3.Database("./db/database.sqlite");

// 3) Criar tabela de usuários (caso ainda não exista)
db.run(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL
  )
`);

// 4) Middlewares
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.use(express.static("public"));

// 5) Rota para cadastro de usuário
app.post("/register", (req, res) => {
  const { username, password } = req.body;
  const hash = bcrypt.hashSync(password, 8);

  db.run(
    "INSERT INTO users (username, password) VALUES (?, ?)",
    [username, hash],
    (err) => {
      if (err) {
        return res.send("Erro: usuário já existe");
      }
      res.send("Cadastro realizado com sucesso! <a href='/'>Voltar ao login</a>");
    }
  );
});

// 6) Rota para login
app.post("/login", (req, res) => {
  const { username, password } = req.body;

  db.get("SELECT * FROM users WHERE username = ?", [username], (err, row) => {
```

```

    if (!row) {
      return res.send("Usuário não encontrado");
    }

    if (!bcrypt.compareSync(password, row.password)) {
      return res.send("Senha incorreta");
    }

    res.send(`Bem-vindo, ${username}!`);
  });
});

// 7) Iniciar o servidor na porta 3000
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});

```

■ Desafio para os alunos

Agora que o sistema funciona com usuário e senha, vamos evoluir adicionando mais informações no cadastro.

■ Tarefa:

1. Alterar a tabela users para ter novos campos: email e dataNascimento. Exemplo em SQL: ALTER TABLE users ADD COLUMN email TEXT; ALTER TABLE users ADD COLUMN dataNascimento TEXT; 2. Editar o formulário register.html e incluir esses novos campos (input type='email' e input type='date'). 3. Atualizar a rota /register no server.js para receber e salvar também esses novos dados. ■ Missão final: cada aluno deve escolher pelo menos 1 campo extra para adicionar no cadastro (telefone, CPF, endereço, etc.) e mostrar funcionando no banco de dados.